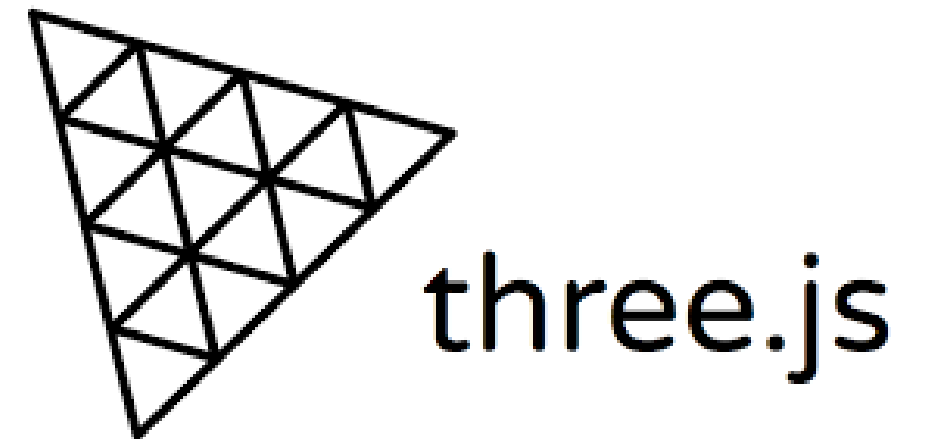


Post processing



Post processing

Le post-processing est le fait d'appliquer un filtre d'image sur le rendu d'une scene pour obtenir un effet particulier.

Ces effets peuvent aller de petits ajustements de couleurs jusqu'à des algorithmes plus complexes qui vont complètement changer le look d'une scene

Before post-processing



After post-processing



Cas d'usage du post processing

- améliorer la visibilité
- renforcer l'identité visuelle
- simuler des effets de caméra ou d'optique
- compenser les limitations de l'affichage en temps réel
- uniformiser l'image
- produire des effet artistiques ou stylisé

Before post-processing



After post-processing



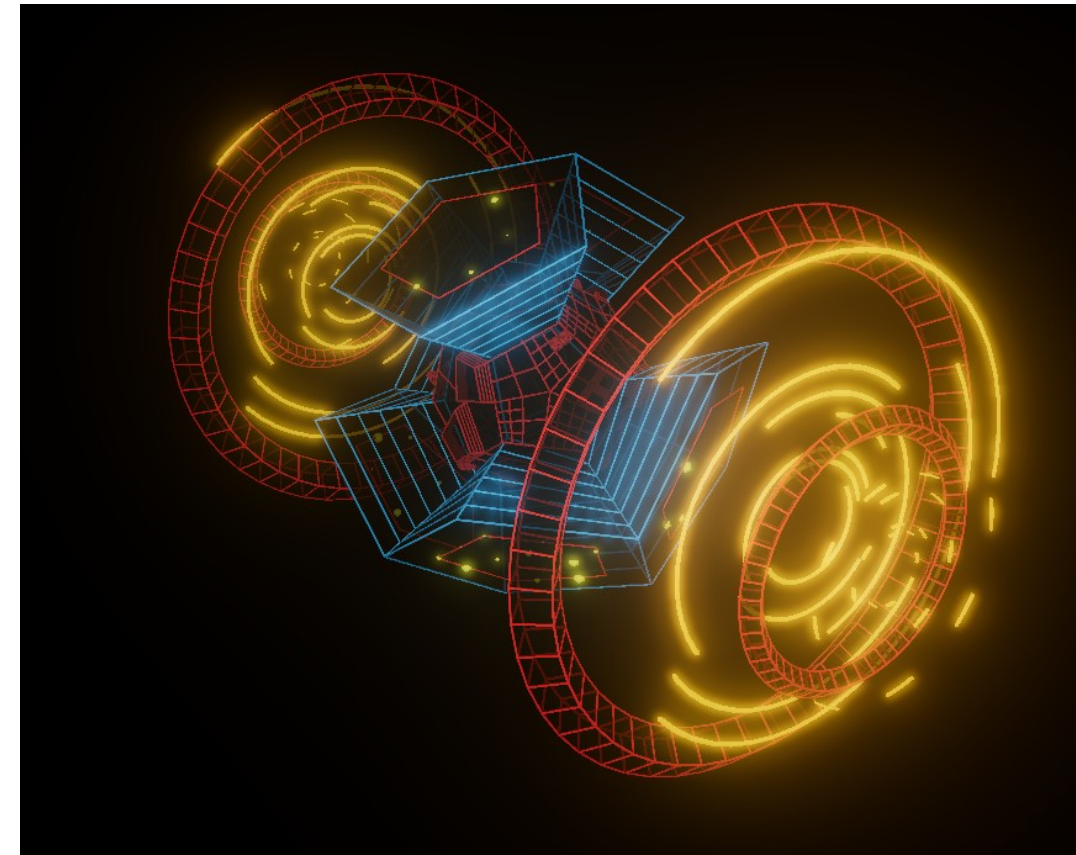
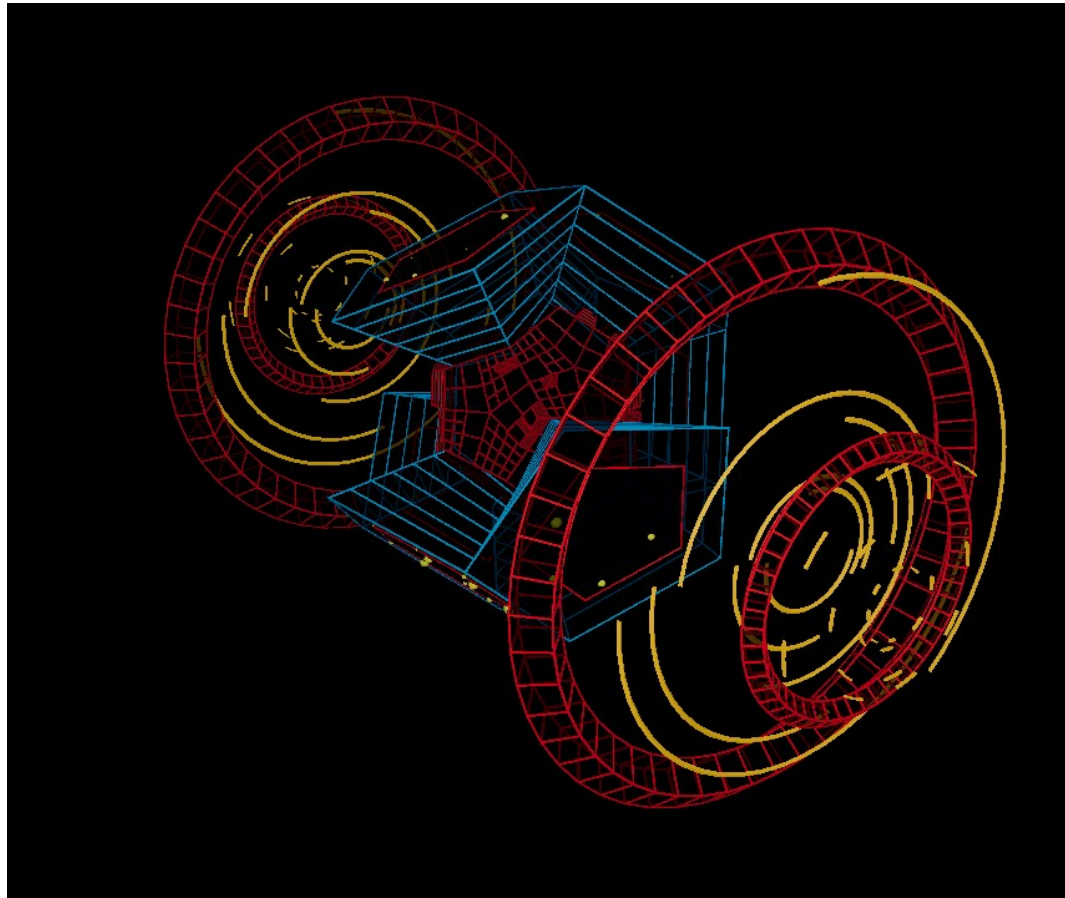
Before post-processing



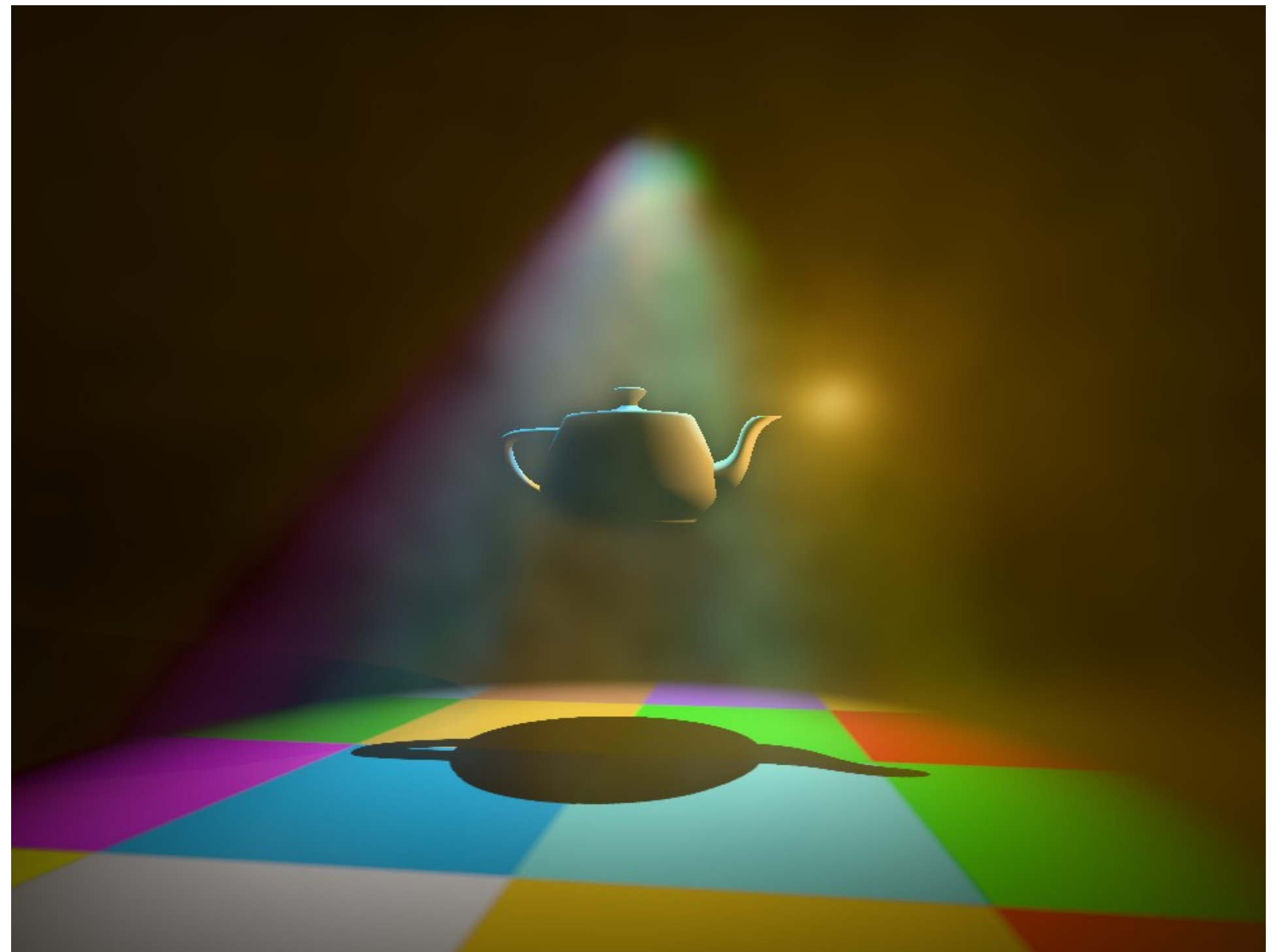
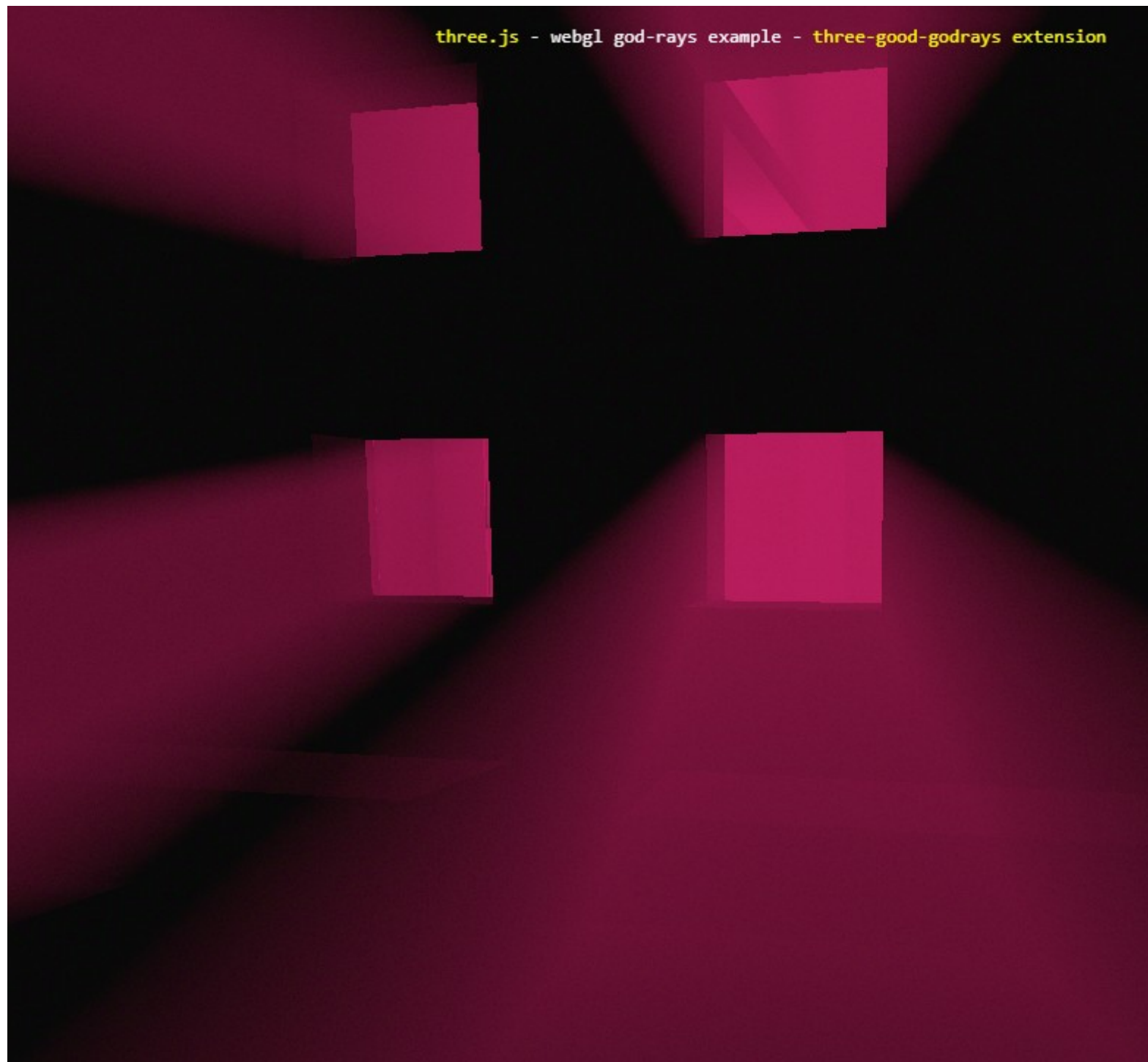
After post-processing



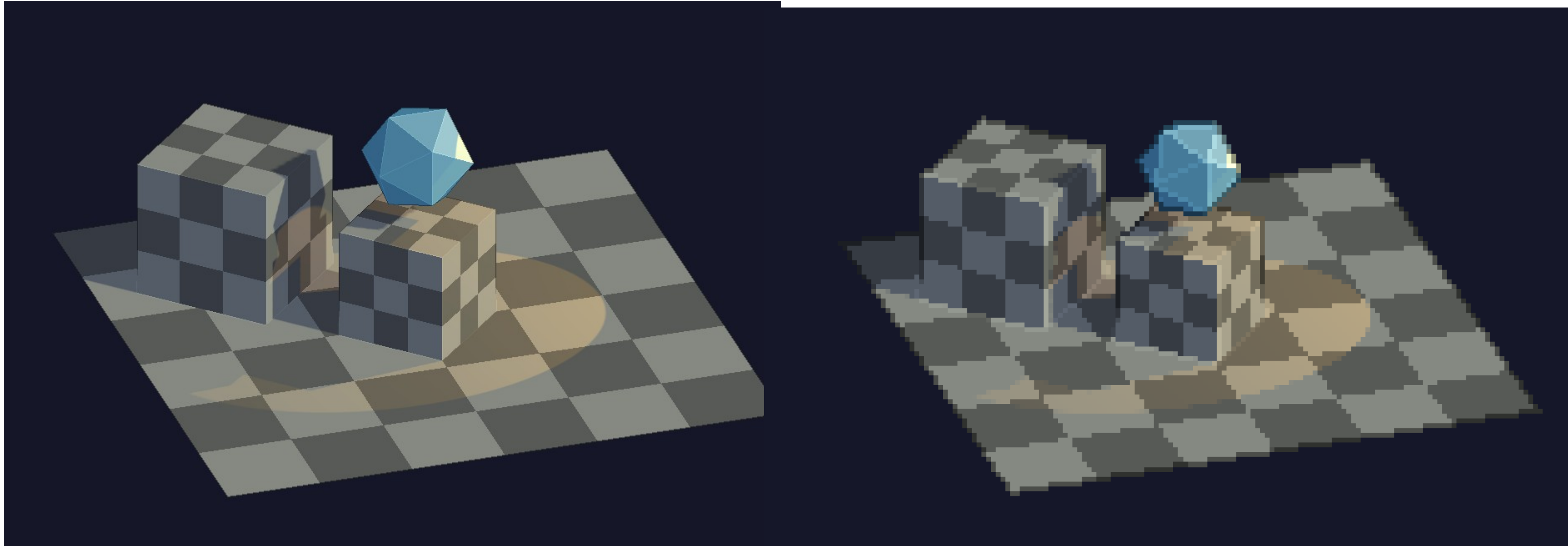
Bloom



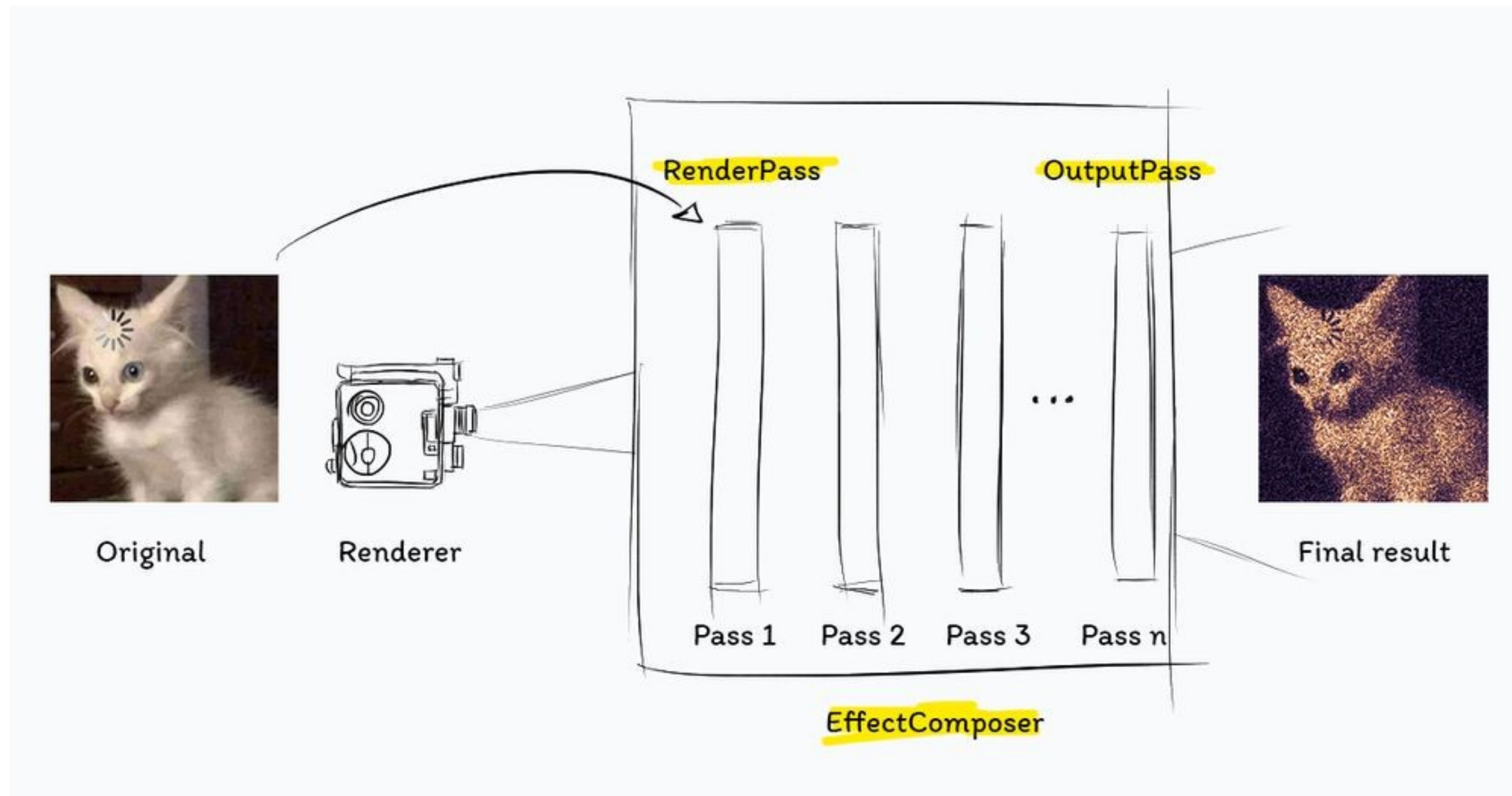
Lumière volumétrique et God Ray



Pixel manipulation



Render Pass



Post processing : Effect Composer

```
import { EffectComposer } from  
  'https://cdn.skypack.dev/three@0.132.2/examples/jsm/postprocessing/EffectComposer.js';
```

```
composer = new EffectComposer(renderer);  
composer.setSize(window.innerWidth, window.innerHeight);
```

Premiere Pass : Render Pass

Les effets de post processing s'appliquent sur le rendu de la scene. Il faut toujours commencer la chaîne de post processing par une RenderPass pour récupérer le rendu de la scene

```
composer = new EffectComposer(renderer);  
composer.setSize(window.innerWidth, window.innerHeight);  
  
var renderPass = new RenderPass(scene, camera);  
composer.addPass(renderPass);
```

Premiere Pass : Render Pass

Les effets de post processing s'appliquent sur le rendu de la scene. Il faut toujours commencer la chaîne de post processing par une RenderPass pour récupérer le rendu de la scene

```
composer = new EffectComposer(renderer);  
composer.setSize(window.innerWidth, window.innerHeight);  
  
var renderPass = new RenderPass(scene, camera);  
composer.addPass(renderPass);
```

Ajout d'étape de post processing

On peut ensuite ajouter autant d'étape de post processing que nécessaire

```
import { UnrealBloomPass } from
  'https://cdn.skypack.dev/three@0.132.2/examples/jsm/postprocessing/UnrealBloomPass.js';

import { RenderPass } from
  'https://cdn.skypack.dev/three@0.132.2/examples/jsm/postprocessing/RenderPass.js';
```

```
composer = new EffectComposer(renderer);
composer.setSize(window.innerWidth, window.innerHeight);

var renderPass = new RenderPass(scene, camera);
composer.addPass(renderPass);

var bloomPass = new UnrealBloomPass(
  new THREE.Vector2(window.innerWidth, window.innerHeight),
  0.87,
  0.82,
  0.5
);

composer.addPass(bloomPass);
```

Render du post processing

Le render se fait maintenant sur **EffectComposer** et non le renderer

```
composer = new EffectComposer(renderer);
composer.setSize(window.innerWidth, window.innerHeight);

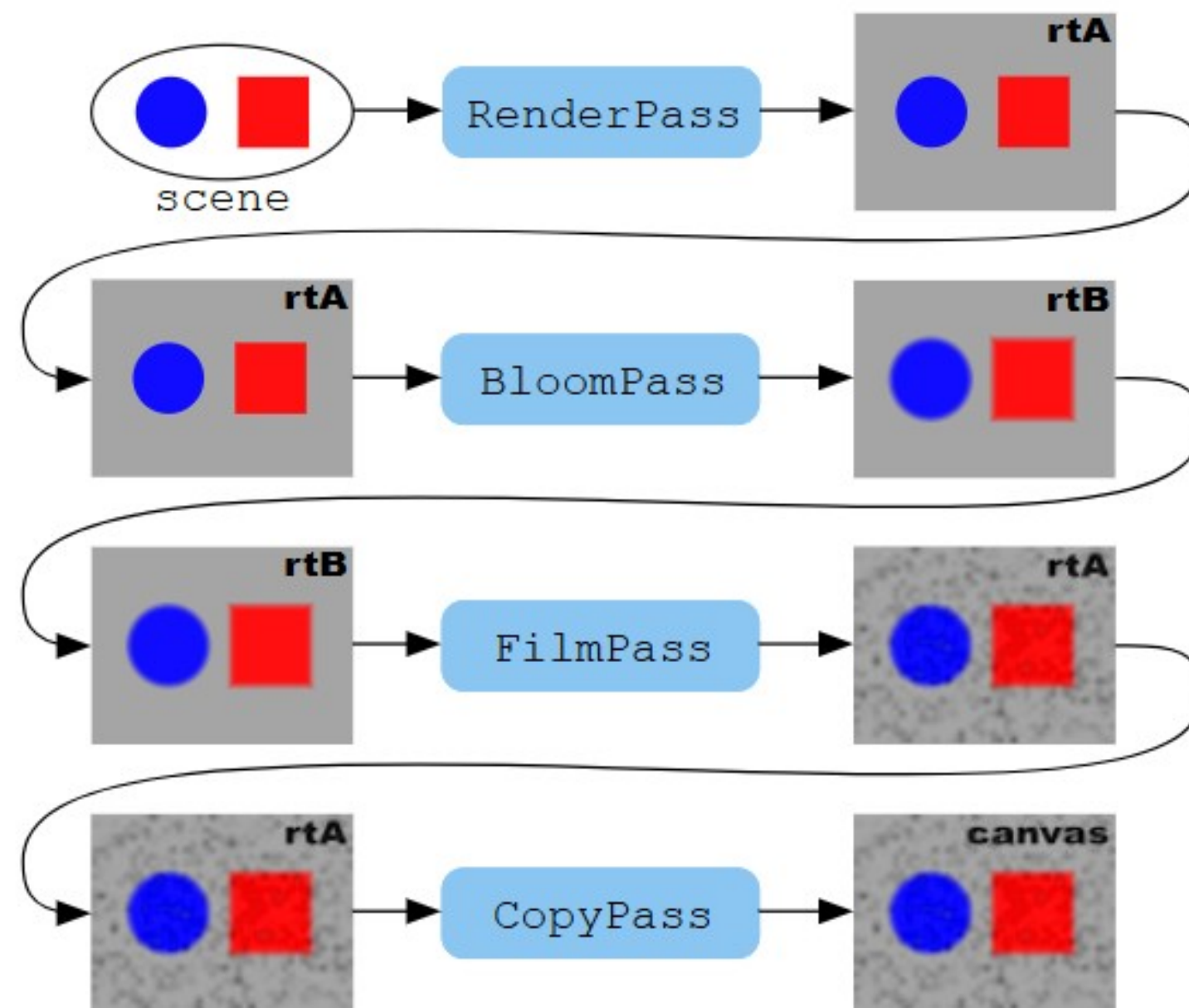
var renderPass = new RenderPass(scene, camera);
composer.addPass(renderPass);

var bloomPass = new UnrealBloomPass(
    new THREE.Vector2(window.innerWidth, window.innerHeight),
    0.87,
    0.82,
    0.5
);

composer.addPass(bloomPass);
```

```
function animate() {
    composer.render(); // instead of renderer.render()
}
```

Pass Order



Post processing et performances

- Chaque pass à un coût : ces algos s'appliquent sur chaque pixel de l'image et dépendent de la résolution du renderer
- Certaines algorithmes font plusieurs passes sur les images ce qui les rendent plus lourdes : Bloom, Antialiasing
- Les algorithmes de post processing dépendent de la résolution du renderer, configurez `renderer.setPixelRatio` correctement. **Faire attention sur mobile, le pixel ratio sur certains mobile est naturellement très élevé**

```
renderer = new THREE.WebGLRenderer({antialias :true});  
renderer.setPixelRatio( window.devicePixelRatio );  
renderer.setSize( WIDTH, HEIGHT );
```

```
renderer.setPixelRatio( 0.5);
```

Resize de la fenetre

Il est conseillé d'adapter la taille du renderer ou du composer lorsque la fenetre est resize pour éviter les glitch graphique

```
window.addEventListener('resize', onWindowResize);

function onWindowResize() {
  let width = window.innerWidth;
  let height = window.innerHeight;

  camera.aspect = width / height;
  camera.updateProjectionMatrix();

  renderer.setSize(width, height);
}
```

```
function onWindowResize() {
  let width = window.innerWidth;
  let height = window.innerHeight;

  camera.aspect = width / height;
  camera.updateProjectionMatrix();

  composer.setSize(width, height);
}
```

Exercices